

21. Convert Expression into Prefix and Postfix

Given Infix Expression

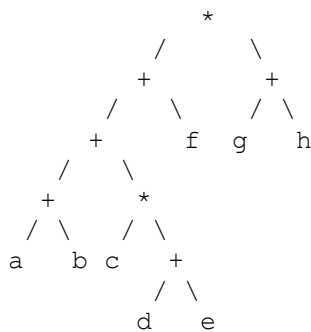
$((a + b) + c * (d + e) + f) * (g + h)$

Step 1: Operator Precedence

Operator Priority

()	Highest
*	Medium
+	Low

Expression Tree



Prefix Expression

Prefix format:

Operator Operand Operand

Therefore:

* + + a b * c + d e f + g h

Answer:

* + + a b * c + d e f + g h

Postfix Expression

Postfix format:

Operand Operand Operator

Therefore:

a b + c d e + * + f + g h + *

Answer:

a b + c d e + * + f + g h + *

Complexity

Conversion using stack:

Time Complexity = $O(n)$

Space Complexity = $O(n)$

22. Recursive Binary Search

Given Array

23, 33, 43, 48, 53, 58, 61, 64, 68, 73, 88

Key:

88

Algorithm

BinarySearch(arr, low, high, key)

1. If low > high return -1
 2. Find mid = (low+high)/2
 3. If arr[mid]==key return mid
 4. If key<arr[mid]
 Search left half
 5. Else
 Search right half
-

Steps

Array indexes:

0 1 2 3 4 5 6 7 8 9 10

23 33 43 48 53 58 61 64 68 73 88

First Call

low=0
high=10

mid=(0+10)/2=5

Element:

arr[5]=58

88 > 58

Search right side.

Second Call

low=6
high=10

mid=(6+10)/2=8

Element:

68

88 > 68

Search right.

Third Call

low=9
high=10

mid=(9+10)/2=9

Element:

73

88 > 73

Search right.

Fourth Call

low=10
high=10

mid=10

Element:

88

Found.

Result

Key 88 found at index position 10

Complexity

Best Case : $O(1)$

Worst Case : $O(\log n)$

23. Circular Queue Operations

MAXSIZE = 6

Initial:

Front = -1
Rear = -1
Count = 0

Circular Queue:

```
+---+---+---+---+---+---+
|   |   |   |   |   |   |
+---+---+---+---+---+---+
 0   1   2   3   4   5
```

Operations

a) Enqueue A,B,C

Front=0
Rear=2

[A] [B] [C] [] [] []

b) Dequeue A

Removed:

A

Queue:

[] [B] [C] [] [] []

Front moves to 1.

c) Enqueue D,E,F

[] [B] [C] [D] [E] [F]

Rear=5

d) Dequeue B,C

Remove:

B, C

Queue:

[] [] [] [D] [E] [F]

e) Enqueue G

Circular movement:

Rear goes to 0

[G] [] [] [D] [E] [F]

f) Dequeue D,E

[G][][][][][F]

g) Enqueue H,I

Circular insertion:

[G][H][I][][][F]

h) Dequeue F,G,H

Final:

[][][I][][][]

Final Status

Front = 2
Rear = 2
Count = 1

Element:

I

24. List ADT Operations

(a)

Operations:

```
L1.append(10)
L1.append(20)
L1.append(15)
```

Linked List:

10 → 20 → 15 → NULL

Current position:

15

(b)

Operations:

```
L2.append(10)
L2.append(20)
L2.append(15)
```

Initial:

10 → 20 → 15

Move to start:

```
^
|
10
```

Insert 39:

39 → 10 → 20 → 15

Next:

39 → 10 → 20 → 15
 ^

Insert 12:

39 → 10 → 12 → 20 → 15

25. Check Validity of Parentheses using Stack

Problem

Check whether the given expression has balanced brackets.

Example:

```
{ (a+b) * [c-d] }
```

Algorithm

```
Step 1: Create an empty stack.
Step 2: Read expression character by character.
Step 3: If opening bracket (, {, [ push into stack.
Step 4: If closing bracket ), }, ]:
        Check stack top.
        If matching bracket → pop.
        Else → invalid.
Step 5: After scanning, if stack is empty → valid.
```

C Program

```
#include<stdio.h>
#include<string.h>

#define MAX 100

char stack[MAX];
int top=-1;

void push(char ch)
{
    stack[++top]=ch;
}

char pop()
{
    return stack[top--];
}

int isValid(char exp[])
{
    for(int i=0; exp[i]!='\0'; i++)
    {
        char ch=exp[i];

        if(ch=='(' || ch=='{' || ch=='[')
        {
            push(ch);
        }
        else if(ch==')' || ch=='}' || ch==']')
        {
            if(top== -1)
                return 0;

            char temp=pop();

            if((ch==')' && temp!='(') ||
               (ch=='}' && temp!='{') ||
               (ch==']' && temp!='['))
                return 0;
        }
    }

    return top== -1;
}
```

Output

Input:

```
{ (a+b) * [c-d] }
```

Output:

Complexity

Time Complexity : $O(n)$

Space Complexity: $O(n)$

26. Palindrome Checking Using Stack and Queue

Algorithm

Step 1: Read characters one by one.
Step 2: Insert characters into stack and queue.
Step 3: Remove characters from stack and queue.
Step 4: Compare both values.
Step 5:
If all characters match $\rightarrow$ Palindrome.
Else $\rightarrow$ Not Palindrome.

Example

Input:

MADAM

Stack:

M
A
D
A
M

Queue:

M A D A M

Comparison:

M=M
A=A
D=D
A=A
M=M

Result:

Complexity

Time Complexity : $O(n)$

Space Complexity: $O(n)$

27. Reorder Singly Linked List

Given List

$L1 \rightarrow L2 \rightarrow L3 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$

Required:

$L1 \rightarrow L_n \rightarrow L2 \rightarrow L_{n-1} \rightarrow L3$

Example

Before:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$

After:

$1 \rightarrow 5 \rightarrow 2 \rightarrow 4 \rightarrow 3$

Algorithm

1. Find middle node.
 2. Reverse second half.
 3. Merge first half and reversed second half.
-

Complexity

Time Complexity : $O(n)$

Space Complexity: $O(1)$

28. Doubly Linked List Insert and Delete

Insert Elements

Elements:

150, 250, 350, 450, 550

Before Deletion

```
NULL
|
150
|
250
|
350
|
450
|
550
|
NULL
```

DLL Diagram:

```
NULL <- 150 <=> 250 <=> 350 <=> 450 <=> 550 -> NULL
```

Delete 150

After deleting first node:

```
NULL <-250 <=>350 <=>450 <=>550 -> NULL
```

Delete 250

Final List:

```
NULL <-350 <=>450 <=>550 -> NULL
```

Steps

1. Locate node.
 2. Change previous node next pointer.
 3. Change next node previous pointer.
 4. Free deleted node.
-

29. Stack Using Array

Stack Size = n

Operations:

- PUSH
 - POP
 - Retrieve mth element
 - Remove even positions
-

Stack Representation

Example:

```
TOP
|
88
66
33
55
22
```

PUSH

```
push(x)
{
    if(top==MAX-1)
        Overflow;

    else
    {
        top++;
        stack[top]=x;
    }
}
```

POP

```
pop()
{
    if(top==-1)
        Underflow;

    else
        return stack[top--];
}
```

Remove Even Positions

Example:

Before:

1 2 3 4 5

Remove even positions:

1 3 5

Complexity

Push : $O(1)$
Pop : $O(1)$
Search : $O(n)$

30. Product Sales Using Linked List

Node Structure

```
struct node
{
    int productID;
    int sales[7];
    struct node *next;
};
```

Operations

Insert Sales

Create node
Store product ID
Store daily sales
Attach node

Total Weekly Sales

Example:

Product:

P101

Sales:

10 20 15 25 30 12 18

Total:

130 units

Highest Sales

Compare total sales of all products.

Display

Example:

Product ID : 101

Mon Tue Wed Thu Fri Sat Sun

10 20 15 25 30 12 18

Complexity

Insertion : $O(1)$

Traversal : $O(n)$

Searching : $O(n)$

31. Postfix Expression Evaluation Using Stack

Given Postfix Expression

5 3 8 * + 4 -

Algorithm

Step 1: Create an empty stack.

Step 2: Scan postfix expression from left to right.

Step 3:

 If operand $\rightarrow$ Push into stack.

 If operator $\rightarrow$ Pop two operands.

 Perform operation.

 Push result back.

Step 4: Final stack value is the answer.

Evaluation Steps

Expression:

5 3 8 * + 4 -

Step 1:

Push 5

Stack:
5

Step 2:

Push 3

Stack:
3
5

Step 3:

Push 8

Stack:
8
3
5

Step 4:

Operator *

Pop:

8 and 3

Operation:

$3 * 8 = 24$

Push 24

Stack:
24
5

Step 5:

Operator +

Pop:

24 and 5

Operation:

$5 + 24 = 29$

Stack:

29

Step 6:

Push 4

Stack:

4

29

Step 7:

Operator -

Operation:

$29 - 4 = 25$

Final Result

Answer = 25

Complexity

Time Complexity : $O(n)$

Space Complexity: $O(n)$

32. Sum of Elements in Singly Linked List

Given List

34 → 77 → 22 → 55 → 66 → 11

Algorithm

Step 1: Start from head node.
Step 2: Add each node data.
Step 3: Move to next node.
Step 4: Print total sum.

Calculation

34 + 77 + 22 + 55 + 66 + 11
= 265

C Code

```
int sum(struct node *head)
{
    int total = 0;

    while(head != NULL)
    {
        total += head->data;
        head = head->next;
    }

    return total;
}
```

Output

Sum = 265

Complexity

Time Complexity : O(n)
Space Complexity: O(1)

33. Difference Between Minimum and Maximum Value

Given List

34 → 77 → 22 → 55 → 66 → 11

Find Values

Maximum:

77

Minimum:

11

Difference:

77 - 11 = 66

C Code

```
int difference(struct node *head)
{
    int max = head->data;
    int min = head->data;

    while(head != NULL)
    {
        if(head->data > max)
            max = head->data;

        if(head->data < min)
            min = head->data;

        head = head->next;
    }

    return max-min;
}
```

Output

Difference = 66

34. Doubly Linked List Forward, Reverse and Average

DLL

```
NULL
|
10 <=> 20 <=> 30 <=> 40
|
NULL
```

Forward Traversal

10 20 30 40

Reverse Traversal

40 30 20 10

Average Calculation

Formula:

Average = Sum of elements / Number of elements

Example:

Sum = 100
Count = 4

Average = 100/4

= 25

Algorithm

1. Traverse DLL.
 2. Add all values.
 3. Count nodes.
 4. Calculate average.
-

Complexity

Time Complexity : O(n)
Space Complexity: O(1)

35. Count Odd and Even Elements in SLL

Given List

77 → 88 → 22 → 11 → 66 → 27 → 55 → 65 → 18 → 10

Odd Numbers

77, 11, 27, 55, 65

Count:

5

Even Numbers

88, 22, 66, 18, 10

Count:

5

C Code

```
void count(struct node *head)
{
    int odd=0, even=0;

    while(head!=NULL)
    {
        if(head->data % 2 == 0)
            even++;
        else
            odd++;

        head=head->next;
    }

    printf("Odd = %d", odd);
    printf("Even = %d", even);
}
```

Output

Odd Count = 5
Even Count = 5

36. Linear Search in Singly Linked List

Key Element

K = 20

Example List:

10 → 15 → 20 → 25 → 30

Algorithm

1. Start from head.
 2. Compare each node value.
 3. If found print YES.
 4. Otherwise print NO.
-

C Code

```
int search(struct node *head,int key)
{
    while(head!=NULL)
    {
        if(head->data==key)
            return 1;

        head=head->next;
    }

    return 0;
}
```

Output

YES

Complexity

Time Complexity : O(n)

37. Reverse DLL Characters

Given DLL

S <=> I <=> M <=> A <=> T <=> S

Forward

SIMATS

Reverse

STAMIS

Algorithm

1. Move pointer to last node.
 2. Traverse using previous pointer.
 3. Print characters.
-

Output

Reverse = STAMIS

38. Reverse SLL Using Stack

Given List

S → I → M → A → T → S

Push into Stack

S
T
A
M
I
S

Pop

Output:

STAMIS

Algorithm

1. Traverse linked list.
2. Push every character into stack.

3. Pop stack elements.
 4. Print reverse string.
-

Complexity

Time Complexity : $O(n)$
Space Complexity: $O(n)$

39. Find Largest Element Using Divide and Conquer

Algorithm

`FindLargest(arr, low, high)`

1. If only one element return it.
 2. Divide array into two halves.
 3. Find largest in both halves.
 4. Return maximum value.
-

Example

Array:

10 25 7 90 45

Divide:

10 25 7 90 45

Largest:

Left = 25
Right = 90

Final:

Largest = 90

Complexity

Time Complexity : $O(n)$
Space Complexity: $O(\log n)$

40. Print Prime Numbers in Singly Linked List

Given List

24 → 21 → 57 → 31 → 12 → 47 → 7 → 38

Check Prime Numbers

Number	Prime
24	No
21	No
57	No
31	Yes
12	No
47	Yes
7	Yes
38	No

Output

Prime Numbers:

31 → 47 → 7

Algorithm

1. Traverse linked list.
 2. Check each number for prime condition.
 3. Print if prime.
-

Complexity

Time Complexity : $O(n\sqrt{m})$

Space Complexity: $O(1)$